



PackUp

API Endpoint Reference

Every endpoint in the PackUp Spring Boot backend — what it does, and exactly when it returns a 400 Bad Request.

Peer-to-peer storage space rental

How to read this document

Every controller in the project is covered below, grouped by resource. For each endpoint you'll find its HTTP method and path, a short description of what it does, and a full list of the conditions under which it responds with 400 Bad Request together with the exact message returned in the response body. Endpoints are tagged CRUD (the five basic create/read/update/delete operations) or Non-CRUD (everything else — the 15+ business-logic endpoints specific to PackUp).

All responses are wrapped in a simple ApiResponse object. A successful call returns 200 OK (the endpoint's own success message or the requested data); a failed one returns 400 Bad Request with a message explaining why, except where noted.

GET	Read data	POST	Create a resource
PUT	Update / trigger an action	DELETE	Remove a resource

Contents

BookingController /api/v1/booking

Booking lifecycle: create, update, cancel, extend, complete, and query bookings. — 13 endpoints

SpaceController /api/v1/space

Space listings: publish, edit, browse, and change availability — plus the AI description generator. — 9 endpoints

UserController /api/v1/user

User accounts: registration, profile updates, and the renter block list. — 6 endpoints

ReviewController /api/v1/review

Reviews: post, edit, delete, and compute average ratings. — 5 endpoints

33 endpoints documented in total.

BookingController /api/v1/booking

Booking lifecycle: create, update, cancel, extend, complete, and query bookings.

BASIC CRUD — 4 ENDPOINTS

GET /api/v1/booking/allBookings

CRUD

Returns every booking in the system.

No Bad Request branch — always returns 200.

Returns 200 with “No Bookings yet” if the table is empty (not an error).

POST /api/v1/booking/createBooking

CRUD

Creates a new booking for a renter on a space.

Returns 400 Bad Request when:

• Request body fails @Valid validation	“(first violated field message)”
• spaceId does not match any space	“No Space with this ID”
• renterId does not match any user	“No renter with this ID”
• The renter is on the space owner's block list	“You are blocked from booking this owner's spaces”
• The space's status is UNAVAILABLE	“This Space is Unavailable”
• The requested dates overlap an existing BOOKED booking	“This space is unavailable in this date”
• Paid amount is below 50% of total price, or exceeds total price	“You have to pay at least half of the total price and not exceeding total price”

Note: Field-level validation also returns 400 (message = the first @Valid constraint violated on the request body, e.g. missing/invalid phone, empty title, price below minimum, etc.).

PUT /api/v1/booking/updateBooking/{id}

CRUD

Updates an existing booking's details.

Returns 400 Bad Request when:

• Request body fails @Valid validation	“(first violated field message)”
• id does not match any booking	“No Booking with this ID”
• Booking status is not BOOKED (already ONGOING/COMPLETED/CANCELLED)	“You can only update a BOOKED booking”
• The booking's start date has already passed	“Too late to update, booking already started”
• New end date is not after the new start date	“End date must be after the start date”
• New dates overlap another booking on the same space	“This end date conflicts with another booking”

Note: Field-level validation also returns 400 (message = the first @Valid constraint violated on the request body, e.g. missing/invalid phone, empty title, price below minimum, etc.).

DELETE /api/v1/booking/deleteBooking/{id}

CRUD

Permanently deletes a booking record.

Returns 400 Bad Request when:

• id does not match any booking	“No Booking with this Id”
---------------------------------	---------------------------

ADDITIONAL ENDPOINTS — 9 ENDPOINTS

GET /api/v1/booking/renterBookings/{renterId}

Non-CRUD

Lists every booking made by a given renter.

Returns 400 Bad Request when:

• renterId does not match any user	"no user with this Id"
• The renter has no bookings	"No Booking with this Id"

GET /api/v1/booking/calculateProfits/{ownerId}

Non-CRUD

Calculates the total profit an owner has earned across all their spaces' completed bookings.

Returns 400 Bad Request when:

• ownerId does not match any user	"No user with this id"
-----------------------------------	------------------------

PUT /api/v1/booking/cancelBooking/{bookingId}/{renterId}

Non-CRUD

Lets a renter cancel their own booking. On success, sends a WhatsApp cancellation notice to the renter.

Returns 400 Bad Request when:

• bookingId does not match any booking	"No booking with this id"
• renterId does not match any user	"No user with this id"
• Booking status is not BOOKED (already ONGOING/COMPLETED/CANCELLED)	"status of booking should be booked, not completed or onGoing booking"
• The booking's start date has already passed	"too late to cancel, booking already started"

GET /api/v1/booking/checkAvailability/{spaceId}/{startDate}/{endDate}

Non-CRUD

Checks whether a space is bookable for a given date range.

Returns 400 Bad Request when:

• spaceId does not match any space	"No space with this id"
• The date range overlaps an existing BOOKED booking	"space is booked on this date"
• startDate is after endDate	"start date should be before end date"
• The space's status is UNAVAILABLE	"This space is currently unavailable"

GET /api/v1/booking/getSpaceBookings/{spaceId}

Non-CRUD

Lists every booking made on a given space.

Returns 400 Bad Request when:

• spaceId does not match any space	"No space with this id"
• The space has no bookings	"No bookings for this space"

PUT /api/v1/booking/extendBooking/{bookingId}/{renterId}/{newEndDate}

Non-CRUD

Lets a renter push a booking's end date further out. On success, sends a WhatsApp confirmation to the renter.

Returns 400 Bad Request when:

• bookingId does not match any booking	"No Booking with this ID"
• renterId is not the renter on this booking	"This is not your booking"
• Booking status is not BOOKED	"You can only extend a booking that is still BOOKED"
• newEndDate is not after the current end date	"New end date must be after the current end date"
• newEndDate is not after the start date	"New end date must be after the start date"
• The extended range overlaps another booking on the same space	"This new end date conflicts with another booking"

PUT /api/v1/booking/updateBookingStatuses

Non-CRUD

Batch job: flips every BOOKED booking whose start date has arrived to ONGOING. Meant to be called on a schedule.

No Bad Request branch — always returns 200.

Always returns 200 with the count of bookings updated — there is no failure condition.

PUT**/api/v1/booking/completeBooking/{bookingId}/{renterId}/{newAmount}****Non-CRUD**

Marks a booking as COMPLETED once the renter pays the remaining balance. On success, sends a WhatsApp thank-you and loyalty-discount notice to the renter.

Returns 400 Bad Request when:

• bookingId does not match any booking	"No Booking with this ID"
• renterId is not the renter on this booking	"This is not your booking"
• renterId does not match any user	"No user with this ID"
• Booking status is already COMPLETED or CANCELLED	"Booking is already completed or cancelled"
• newAmount is less than the remaining balance owed	"You still have a remaining amount to pay"
• newAmount is more than the remaining balance owed	"Amount exceeds the remaining balance"

GET**/api/v1/booking/loyaltyDiscountUsers/{ownerId}****Non-CRUD**

Lists renter IDs who qualify for a loyalty discount on this owner's spaces (3+ completed bookings).

Returns 400 Bad Request when:

• ownerId does not match any user	"No owner with this id"
-----------------------------------	-------------------------

Returns 200 with "No renters eligible for discount yet" if the list is empty (not an error).

SpaceController /api/v1/space

Space listings: publish, edit, browse, and change availability — plus the AI description generator.

BASIC CRUD — 4 ENDPOINTS

GET /api/v1/space/allSpaces

CRUD

Returns every storage space in the system.

Returns 400 Bad Request when:

- | | |
|-----------------------------------|-------------|
| • No spaces exist in the database | “No Spaces” |
|-----------------------------------|-------------|

Note: Unlike other list endpoints, an empty result here is treated as a 400, not a 200.

POST /api/v1/space/createSpace

CRUD

Publishes a new storage space listing for an owner.

Returns 400 Bad Request when:

- | | |
|--|----------------------------------|
| • Request body fails @Valid validation | “(first violated field message)” |
| • ownerId does not match any user | “No Space Owner With This ID !” |

Note: Field-level validation also returns 400 (message = the first @Valid constraint violated on the request body, e.g. missing/invalid phone, empty title, price below minimum, etc.).

PUT /api/v1/space/updateSpace/{id}

CRUD

Updates an existing space's details.

Returns 400 Bad Request when:

- | | |
|---|---------------------------------------|
| • Request body fails @Valid validation | “(first violated field message)” |
| • id does not match any space | “No Space With This ID !” |
| • The space's ownerId does not match any user | “No Space Owner With This ID !” |
| • The caller is not the owner of this space | “You are not the owner of this space” |

Note: Field-level validation also returns 400 (message = the first @Valid constraint violated on the request body, e.g. missing/invalid phone, empty title, price below minimum, etc.).

DELETE /api/v1/space/deleteSpace/{id}

CRUD

Permanently deletes a space listing.

Returns 400 Bad Request when:

- | | |
|-------------------------------|-------------------------|
| • id does not match any space | “No space with this Id” |
|-------------------------------|-------------------------|

ADDITIONAL ENDPOINTS — 5 ENDPOINTS

GET /api/v1/space/ownerSpaces/{ownerId}

Non-CRUD

Lists every space owned by a given user.

Returns 400 Bad Request when:

- | | |
|-----------------------------------|---------------------------|
| • ownerId does not match any user | “No user with this Id” |
| • The user owns no spaces | “No spaces for this user” |

GET /api/v1/space/citySpace/{city}

Non-CRUD

Lists every space located in a given city.

Returns 400 Bad Request when:

- No spaces exist in that city
- | “No spaces in this location”

GET

/api/v1/space/budgetSpace/{budget}

Non-CRUD

Lists every space priced at or under a given daily budget.

Returns 400 Bad Request when:

- No spaces fall within the budget
- | “No spaces within this range”

PUT

/api/v1/space/changeStatus/{spaceId}/{ownerId}

Non-CRUD

Toggles a space between AVAILABLE and UNAVAILABLE. Only the owner can do this.

Returns 400 Bad Request when:

• spaceId does not match any space	“No space with this id”
• ownerId does not match any user	“No owner with this id”
• The owner owns no spaces at all	“owner doesn't own any spaces yet”
• The owner does not own this specific space	“owner doesn't own the space”

POST

/api/v1/space/generateDescription

Non-CRUD

AI feature: generates a bilingual (Arabic + English) marketing description for a space via the Gemini API, from its title, city, address, price and size.

No Bad Request branch — always returns 200.

Returns 500 (not 400) with “Failed to generate description, please try again” if the Gemini call fails or times out; there is no explicit input-validation branch.

UserController /api/v1/user

User accounts: registration, profile updates, and the renter block list.

BASIC CRUD — 4 ENDPOINTS

GET /api/v1/user/allUsers

CRUD

Returns every registered user.

No Bad Request branch — always returns 200.

Returns 200 with “No Users” if the table is empty (not an error).

POST /api/v1/user/createUser

CRUD

Registers a new user (owner or renter) with a phone number and plaintext password.

Returns 400 Bad Request when:

• Request body fails @Valid validation	“(first violated field message)”
• The phone number is already registered	“Phone number already exists”

Note: Field-level validation also returns 400 (message = the first @Valid constraint violated on the request body, e.g. missing/invalid phone, empty title, price below minimum, etc.).

PUT /api/v1/user/updateUser/{id}

CRUD

Updates an existing user's profile.

Returns 400 Bad Request when:

• Request body fails @Valid validation	“(first violated field message)”
• id does not match any user	“No user with this Id”
• The new phone number is already used by another user	“Phone number already exists bu another user”

Note: Field-level validation also returns 400 (message = the first @Valid constraint violated on the request body, e.g. missing/invalid phone, empty title, price below minimum, etc.).

DELETE /api/v1/user/deleteUser/{id}

CRUD

Permanently deletes a user account.

Returns 400 Bad Request when:

• id does not match any user	“No user with this Id”
------------------------------	------------------------

ADDITIONAL ENDPOINTS — 2 ENDPOINTS

PUT /api/v1/user/blockRenter/{ownerId}/{blockedRenter}

Non-CRUD

Lets an owner block a renter from booking any of their spaces.

Returns 400 Bad Request when:

• ownerId does not match any user	“No owner with this id”
• blockedRenter does not match any user	“No renter with this id”
• The renter is already on this owner's block list	“This renter is already blocked”
• The renter has an active (BOOKED or ONGOING) booking with this owner	“Cannot block a renter with active booking (BOOKED or ONGOING)”

GET /api/v1/user/getBlockedRenters/{ownerId}

Non-CRUD

Lists the renter IDs a given owner has blocked.

Returns 400 Bad Request when:

- ownerId does not match any user | “No owner with this id”

Returns 200 with “This owner has not blocked any renters” if the list is empty (not an error).

ReviewController /api/v1/review

Reviews: post, edit, delete, and compute average ratings.

BASIC CRUD — 4 ENDPOINTS

GET /api/v1/review/allReviews

CRUD

Returns every review in the system.

No Bad Request branch — always returns 200.

Returns 200 with “No reviews” if the table is empty (not an error).

POST /api/v1/review/createReview

CRUD

Posts a review from one user to another about a completed booking.

Returns 400 Bad Request when:

• Request body fails @Valid validation	“(first violated field message)”
• bookingId does not match any booking	“No Booking with this id”
• reviewerId or revieweeId does not match any user	“No User with this id”
• The caller's id does not match the reviewer on the booking	“Your id is incorrect”
• The booking's status is not COMPLETED	“You can only review a completed booking”
• reviewerId is the same as revieweeId	“reviewer id is the same as reviewee id”

Note: Field-level validation also returns 400 (message = the first @Valid constraint violated on the request body, e.g. missing/invalid phone, empty title, price below minimum, etc.).

PUT /api/v1/review/updateReview/{id}

CRUD

Edits an existing review's rating or comment.

Returns 400 Bad Request when:

• Request body fails @Valid validation	“(first violated field message)”
• id does not match any review	“No Review with this id”
• The review's bookingId does not match any booking	“No Booking with this id”
• reviewerId or revieweeId does not match any user	“No User with this id”
• The caller's id does not match the original reviewer	“Your id is incorrect”
• The caller did not author this review	“you didn't write this review”
• reviewerId is the same as revieweeId	“reviewer id is the same as reviewee id”

Note: Field-level validation also returns 400 (message = the first @Valid constraint violated on the request body, e.g. missing/invalid phone, empty title, price below minimum, etc.).

DELETE /api/v1/review/deleteReview/{id}

CRUD

Permanently deletes a review.

Returns 400 Bad Request when:

• id does not match any review	“No review with this Id”
--------------------------------	--------------------------

ADDITIONAL ENDPOINTS — 1 ENDPOINTS

GET /api/v1/review/getAvgRating/{id}

Non-CRUD

Calculates a user's average rating across all reviews they've received.

Returns 400 Bad Request when:

• id does not match any user	“No user with this Id”
------------------------------	------------------------

